

程序鉴别材料 - 法院保全案件管理系统

```
# ===== 第 1 页 =====

# 法院保全案件管理系统 - 程序鉴别材料
# 源程序前30页和后30页（示例）

# ===== 第 1 页 =====
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
import os, hashlib, json

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv('DB_URL', 'mysql+pymysql://user:pass@localhost/court_baquan')
# ===== 第 2 页 =====
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class Case(db.Model):
    __tablename__ = 'bq_cases'
    id = db.Column(db.Integer, primary_key=True)
    case_no = db.Column(db.String(64), unique=True, nullable=False)
    case_type = db.Column(db.String(32), default='财产保全')
    court_id = db.Column(db.String(16), nullable=False)
    applicant_name = db.Column(db.String(128), nullable=False)
    applicant_idcard = db.Column(db.String(32))
    applicant_phone = db.Column(db.String(32))
# ===== 第 3 页 =====
    respondent_name = db.Column(db.String(128), nullable=False)
    respondent_idcard = db.Column(db.String(32))
    respondent_phone = db.Column(db.String(32))
    amount = db.Column(db.Numeric(16,2))
    reason = db.Column(db.Text)
    status = db.Column(db.String(16), default='draft')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class Property(db.Model):
    __tablename__ = 'bq_properties'
    id = db.Column(db.Integer, primary_key=True)
# ===== 第 4 页 =====
    case_id = db.Column(db.Integer, db.ForeignKey('bq_cases.id'))
    property_type = db.Column(db.String(32))
    property_desc = db.Column(db.Text)
    property_value = db.Column(db.Numeric(16,2))
    evidence_url = db.Column(db.String(256))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

class Ruling(db.Model):
    __tablename__ = 'bq_rulings'
```

```

    id = db.Column(db.Integer, primary_key=True)
    case_id = db.Column(db.Integer, db.ForeignKey('bq_cases.id'))
    ruling_no = db.Column(db.String(64))
# ===== 第 5 页 =====
    ruling_type = db.Column(db.String(32))
    ruling_content = db.Column(db.Text)
    issued_at = db.Column(db.DateTime)
    judge_name = db.Column(db.String(64))

# ===== 第 2 页 =====
@app.route('/api/cases', methods=['POST'])
def create_case():
    data = request.get_json()
    required = ['court_id', 'applicant_name', 'respondent_name']
    for f in required:
        if not data.get(f):
# ===== 第 6 页 =====
def api_page_6_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code': 400, 'msg': 'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code': 404, 'msg': '案件不存在'}
    return {'code': 200, 'data': case.to_dict()}

def validate_page_6(form):
    required = ['court_id', 'applicant_name', 'respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok': False, 'msg': f+' 缺失'}
    return {'ok': True}
# ===== 第 7 页 =====
def api_page_7_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code': 400, 'msg': 'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code': 404, 'msg': '案件不存在'}
    return {'code': 200, 'data': case.to_dict()}

def validate_page_7(form):
    required = ['court_id', 'applicant_name', 'respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok': False, 'msg': f+' 缺失'}
    return {'ok': True}
# ===== 第 8 页 =====
def api_page_8_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:

```

```

        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_8(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 9 页 =====
def api_page_9_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_9(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 10 页 =====
def api_page_10_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_10(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 11 页 =====
def api_page_11_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}

```

```

        return {'code':200,'data':case.to_dict()}

def validate_page_11(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 12 页 =====
def api_page_12_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':': case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':': 案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_12(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 13 页 =====
def api_page_13_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':': case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':': 案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_13(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 14 页 =====
def api_page_14_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':': case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':': 案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_14(form):
    required = ['court_id','applicant_name','respondent_name']

```

```

        for f in required:
            if not form.get(f):
                return {'ok':False,'msg':f+' 缺失'}
        return {'ok':True}
# ===== 第 15 页 =====
def api_page_15_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_15(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}
# ===== 第 16 页 =====
def api_page_16_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_16(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}
# ===== 第 17 页 =====
def api_page_17_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_17(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

```

```

# ===== 第 18 页 =====
def api_page_18_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_18(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 19 页 =====
def api_page_19_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_19(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 20 页 =====
def api_page_20_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_20(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 21 页 =====
def api_page_21_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:

```

```

        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_21(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 22 页 =====
def api_page_22_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_22(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 23 页 =====
def api_page_23_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_23(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 24 页 =====
def api_page_24_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}

```

```

        return {'code':200,'data':case.to_dict()}

def validate_page_24(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 25 页 =====
def api_page_25_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_25(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 26 页 =====
def api_page_26_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_26(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

# ===== 第 27 页 =====
def api_page_27_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_27(form):
    required = ['court_id','applicant_name','respondent_name']

```



```

        for f in required:
            if not form.get(f):
                return {'ok':False,'msg':f+' 缺失'}
        return {'ok':True}
# ===== 第 28 页 =====
def api_page_28_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_28(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}
# ===== 第 29 页 =====
def api_page_29_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_29(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}
# ===== 第 30 页 =====
def api_page_30_handler(request):
    case_id = request.args.get('case_id')
    if not case_id:
        return {'code':400,'msg':'case_id必填'}
    case = Case.query.get(case_id)
    if not case:
        return {'code':404,'msg':'案件不存在'}
    return {'code':200,'data':case.to_dict()}

def validate_page_30(form):
    required = ['court_id','applicant_name','respondent_name']
    for f in required:
        if not form.get(f):
            return {'ok':False,'msg':f+' 缺失'}
    return {'ok':True}

```

```

# ===== 第 1 页（后30页） =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows

# ===== 第 2 页（后30页） =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 3 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone]
)
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.stat
us])
    return rows
# ===== 第 4 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone]
)
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.stat
us])

```

```

        return rows

# ===== 第 5 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows

# ===== 第 6 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 7 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 8 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 9 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 10 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 11 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 12 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 13 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 14 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```



```

        return rows
# ===== 第 15 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 16 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows

# ===== 第 17 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows

# ===== 第 18 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 19 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 20 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 21 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 22 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 23 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 24 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 25 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 26 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows
# ===== 第 27 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows
# ===== 第 28 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])
        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```

```

        return rows

# ===== 第 29 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])
    return rows

# ===== 第 30 页 (后30页) =====
# 后端任务调度与邮件通知模块
from apscheduler.schedulers.background import BackgroundScheduler
from flask_mail import Mail, Message
mail = Mail(app)
scheduler = BackgroundScheduler()

def remind_pending_cases():
    pending = Case.query.filter_by(status='pending').all()
    for c in pending:
        msg = Message('保全案件提醒', sender='court@example.com', recipients=[c.applicant_phone])

        msg.body = f'您的案件 {c.case_no} 待处理, 请及时查看。'
        mail.send(msg)

scheduler.add_job(remind_pending_cases, 'cron', hour=9)
scheduler.start()

# 数据统计与报表导出
def export_case_report(start, end):
    cases = Case.query.filter(Case.created_at.between(start, end)).all()
    rows = [['案号', '申请人', '被申请人', '标的金额', '状态']]
    for c in cases:
        rows.append([c.case_no, c.applicant_name, c.respondent_name, str(c.amount), c.status])

```



```
return rows
```